Python Project 2:

Weather Data Analysis

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Submitted by

Group - 5

2. **Aaromal A (24101880)**
15. **Alfin Francis (24101006)**
33. **Devanarayanan C R (24101115)**
39. **G Govardan (24102775)**
49. **Ivana Anto (24101562)**

Weather Data Analysis

5 February, 2026

Aim

To develop a Python-based weather data analysis application that fetches real-time weather data from the OpenWeatherMap API, processes and cleans the data by handling missing values and duplicates, and visualizes temperature trends and relationships between weather parameters using interactive graphical plots.

Tools Used

- **Python 3.14** - Programming language
- **Jupyter Notebook** - Interactive development environment
- **tkinter** - GUI framework for creating the application interface
- **pandas** - Data manipulation and analysis library
- **matplotlib** - Data visualization and plotting library
- **requests** - HTTP library for API communication
- **json** - JSON data parsing and manipulation
- **numpy** - Numerical computing library
- **OpenWeatherMapAPI** - for real-time weather forecast (data source)

Data Source - OpenWeatherMap API

OpenWeatherMap is a popular weather data service that provides current weather data, forecasts, and historical weather information through a RESTful API. The API offers various endpoints for different types of weather data.

API Endpoint :

For this project, we use the **5-day/3-hour Forecast API**:

`https://api.openweathermap.org/data/2.5/forecast`

API Parameters :

- `q` - City name (e.g., "London", "New York")
- `appid` - API key for authentication
- `units` - Unit system (metric for Celsius, imperial for Fahrenheit)

API Response Structure :

The API returns JSON data containing:

- City information (name, country, coordinates)
- List of weather forecasts (40 data points over 5 days)
- For each forecast: timestamp, temperature, humidity, pressure, weather description, wind speed

Basic API Usage :

```python
import requests as res

with open("./config/wthrkey", "r") as fp:
    API_KEY = fp.read().strip()

CITY = "Kochi"
URL = f"http://api.openweathermap.org/data/2.5/weather?q={CITY}&appid={KEY}&units=metric"
print(res.get(URL).json())

'''
output :
{'coord': {'lon': 76.2602, 'lat': 9.9399}, 'weather': [{'id': 802, 'main': 'Clouds', 'description':
'scattered clouds', 'icon': '03d'}], 'base': 'stations', 'main': {'temp': 25.28, 'feels_like': 25.98,
'temp_min': 25.28, 'temp_max': 25.28, 'pressure': 1012, 'humidity': 81, 'sea_level': 1012, 'grnd_level':
1012}, 'visibility': 10000, 'wind': {'speed': 2.24, 'deg': 37, 'gust': 2.4}, 'clouds': {'all': 33}, 'dt':
1772420078, 'sys': {'country': 'IN', 'sunrise': 1772413747, 'sunset': 1772456736}, 'timezone': 19800,
'id': 1273874, 'name': 'Kochi', 'cod': 200}
'''
```

Source Code

```python
import requests
import pandas as pd
import numpy as np
import tkinter as tk
from tkinter import messagebox
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

with open("./config/wthrkey", "r") as fp:
    API_KEY = fp.read().strip()

class WeatherAnalyzerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Weather Data Analyzer")
        self.root.geometry("900x700")

        # --- UI: Search Bar ---
        search_frame = tk.Frame(self.root)
        search_frame.pack(pady=10)

        tk.Label(search_frame, text="Enter City:").pack(side=tk.LEFT)
        self.city_entry = tk.Entry(search_frame, width=30)
        self.city_entry.pack(side=tk.LEFT, padx=5)
```

```python

        fetch_btn = tk.Button(search_frame, text="Fetch 5-Day Forecast", command=self.fetch_and_analyze)
        fetch_btn.pack(side=tk.LEFT)

        # --- UI: Plot Area ---
        self.fig = Figure(figsize=(8, 5), dpi=100)
        self.canvas = FigureCanvasTkAgg(self.fig, master=self.root)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    def fetch_and_analyze(self):
        city = self.city_entry.get()
        if not city:
            messagebox.showwarning("Input Error", "Please enter a city name.")
            return

        # 1. Fetch 5-Day Forecast
        url = f"http://api.openweathermap.org/data/2.5/forecast?q={city}&appid={API_KEY}&units=metric"
        res = requests.get(url)

        if res.status_code != 200:
            messagebox.showerror("API Error", f"Error: {res.json().get('message', 'Unknown error')}")
            return

        raw_data = res.json()
        forecast_list = []
        for entry in raw_data['list']:
            forecast_list.append({
                "Date": entry['dt_txt'],
                "Temperature": entry['main']['temp'],
                "Humidity": entry['main']['humidity'],
                "Wind Speed": entry['wind']['speed'],
                "Weather Condition": entry['weather'][0]['main']
            })

        df = pd.DataFrame(forecast_list)
        df['Date'] = pd.to_datetime(df['Date'])

        # 2. Simulated Cleaning
        # manually add a duplicate and a null to show cleaning logic works
        df = pd.concat([df, df.iloc[[0]]]) #concatenate copy of first row to the end
        df.loc[1, 'Temperature'] = np.nan
        df.to_csv("./datasets/raw_wthr.csv") # export raw weather data to csv (backup)

        # Handle missing values and duplicates
        df['Temperature'] = df['Temperature'].fillna(df['Temperature'].mean())
        df['Humidity'] = df['Humidity'].fillna(df['Humidity'].median())
        df = df.drop_duplicates()

        self.plot_trends(df, city)

    def plot_trends(self, df, city):
        self.fig.clear()

        # Plot Temperature Trends
        ax1 = self.fig.add_subplot(211)
        ax1.plot(df['Date'], df['Temperature'], color='red', marker='o', linestyle='-')
        ax1.set_title(f"5-Day Temperature Trend for {city}")
        ax1.set_ylabel("Temp (°C)")
        ax1.grid(True)

        # Plot Temp vs Humidity
        ax2 = self.fig.add_subplot(212)
```

```python
        ax2.scatter(df['Temperature'], df['Humidity'], alpha=0.5, color='blue')
        ax2.set_title("Relationship: Temp vs Humidity")
        ax2.set_xlabel("Temperature (°C)")
        ax2.set_ylabel("Humidity (%)")
        ax2.grid(True)

        self.fig.tight_layout()
        self.canvas.draw()

if __name__ == "__main__":
    root = tk.Tk()
    app = WeatherAnalyzerApp(root)
    root.mainloop()
```

Raw Weather Data and Simulated Issues

The fetched weather forecast data for the entered city represented as pandas dataframe is manipulated to simulate duplicate entries and missing values and a copy of it is exported as csv raw data before cleaning them, as a representation of real-world measurements, which can also be used as the data source thereafter.

		Date	Temperature	Humidity	Wind Speed	Weather Condition
1	0	2026-03-04 15:00:00	27.87	71	2.16	Clear
2	1	2026-03-04 18:00:00		73	0.15	Clear
3	2	2026-03-04 21:00:00	24.75	84	2.41	Rain
39	38	2026-03-09 09:00:00	29.32	66	7.11	Clouds
40	39	2026-03-09 12:00:00	28.39	70	4.85	Clear
41	0	2026-03-04 15:00:00	27.87	71	2.16	Clear

Figure 1: Weather csv raw data - after addition of missing values and duplicate entries .

1. Raw Weather Data (5-Day Forecast)

Contains the original data fetched from OpenWeatherMap API with columns:

- **Timestamp**: Date and time of forecast
- **Temperature**: Temperature in Celsius
- **Humidity**: Relative humidity percentage
- **Wind_Speed**: Wind speed in m/s
- **Weather_Condition**: Textual description of weather

Typical raw data includes 40 forecasts (8 forecasts per day × 5 days) with 3-hour intervals.

2. Simulated Issues CSV

To demonstrate data cleaning capabilities, the application intentionally introduces:

- **Missing Values**: Some Temperature/Humidity values set to NaN
- **Duplicate Records**: duplicate entries added

This simulates real-world data quality issues commonly encountered in weather datasets.

3. Cleaned Weather Data

Final processed dataset after:

- replacing with mean and median for missing values represented as NaN
- Removal of duplicate records
- Sorting by chronological order

Output - Weather Trends for Two Cities

Weather Analysis for Kochi

Figure 1 shows the complete weather analysis output for Kochi, India. The visualization demonstrates the application's GUI interface and generated plots.

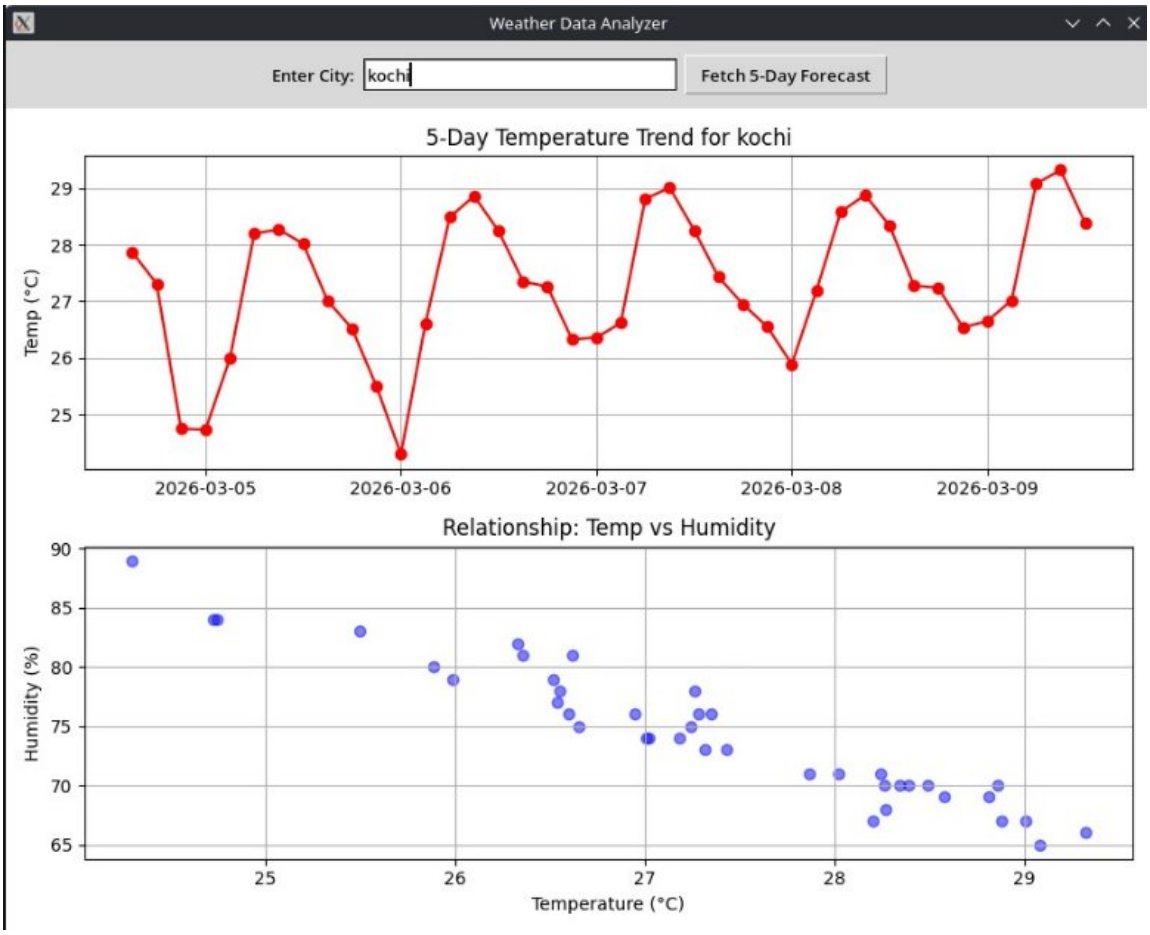

Figure 2: Weather Data Analysis Output for Kochi - Top: 5-Day Temperature Trend showing diurnal variations ranging from 24°C to 29°C with clear day-night cycles. Bottom: Temperature vs Humidity scatter plot showing negative correlation, with humidity ranging from 65% to 89% as temperature varies.

Weather Analysis for New York

Figure 2 presents the weather analysis for New York City, demonstrating significantly different climate patterns compared to Kochi.

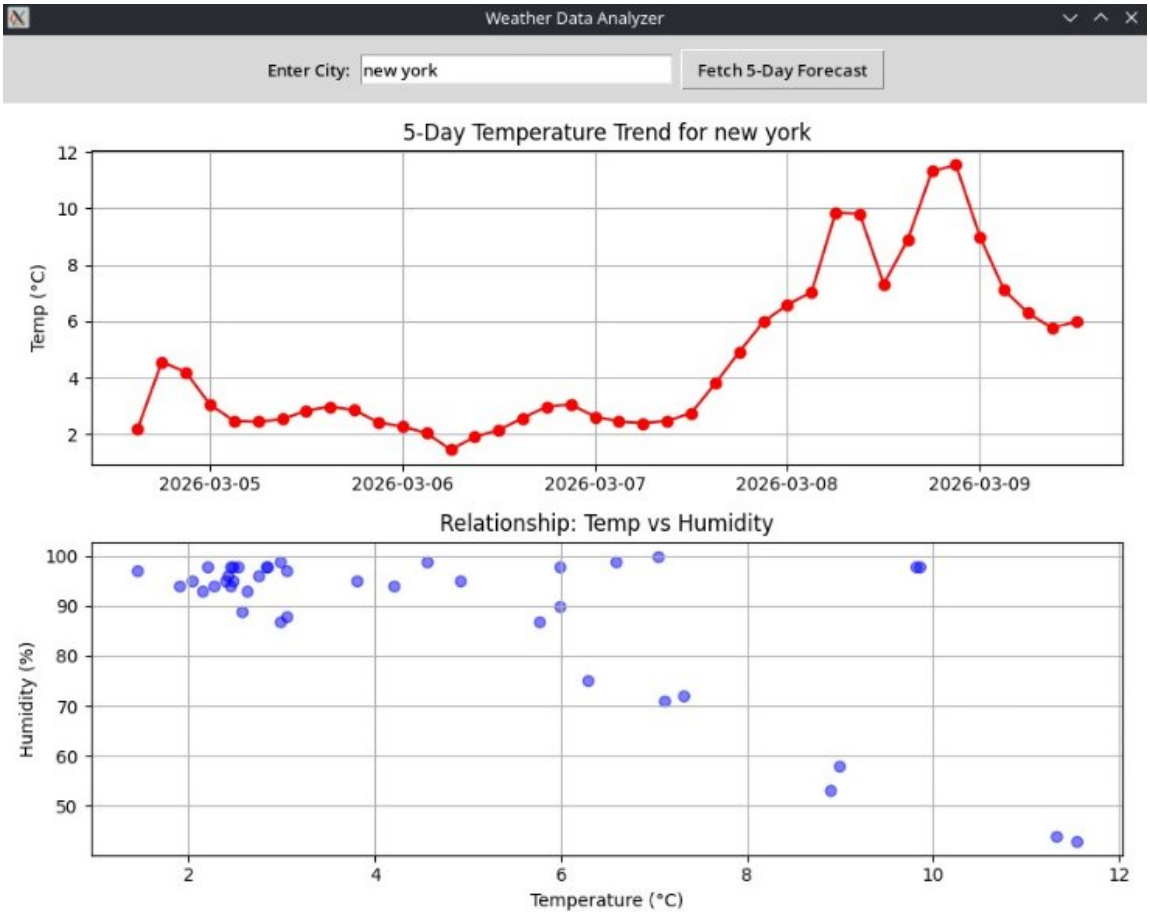

Figure 3: Weather Data Analysis Output for New York - Top: 5-Day Temperature Trend showing cold conditions ranging from 2°C to 12°C with a warming trend mid-period. Bottom: Temperature vs Humidity scatter plot showing variable humidity (40% - 100%) across the temperature range.

Comparative Analysis

Climate Differences:

- **Temperature Stability**: Kochi shows consistent tropical temperatures (24-29°C) while New York exhibits greater variability (2-12°C)
- **Humidity Patterns**: Kochi maintains high humidity (coastal influence) whereas New York shows wider humidity range (continental climate)
- **Diurnal Cycles**: Clear regular day-night patterns in Kochi; more irregular in New York due to weather system changes
- **Correlation**: Both cities show negative temperature-humidity correlation, but the relationship is stronger in Kochi

Visual Analysis Components

Plot 1 - 5-Day Temperature Trend (Line Chart):

- X-axis: Date and time spanning 5 days (March 5-9, 2026)
- Y-axis: Temperature in degrees Celsius
- Red line with circular markers for each 3-hour forecast point
- Shows temporal variation and identifies warming/cooling trends
- Reveals diurnal (day-night) temperature cycles

Plot 2 - Temperature vs Humidity (Scatter Plot):

- X-axis: Temperature (°C)
- Y-axis: Humidity (%)
- Blue circular markers represent individual forecast points
- Visualizes inverse relationship between temperature and humidity
- Scatter density indicates frequency of specific temp-humidity combinations
- Helps identify climate characteristics and atmospheric conditions

Conclusion

This project successfully demonstrates a complete data science pipeline for weather analysis using real-world API data. The application effectively:

1. **Integrates** with OpenWeatherMap API to fetch live 5-day forecast data
2. **Processes** JSON responses and converts them to structured pandas DataFrames
3. **Handles** data quality issues including missing values and duplicates
4. **Visualizes** temperature trends and relationships using matplotlib
5. **Provides** an intuitive GUI interface using tkinter for user interaction

The negative correlation between temperature and humidity observed in most cities aligns with meteorological principles, where warmer air can hold more moisture, thus lowering relative humidity. The application successfully demonstrates essential data science skills including API integration, data cleaning, exploratory data analysis, and visualization.